

Performance Testing

Date	03 October 2026
Team ID	SWTID-2026-5999
Project Name	PocketSmart AI — AI-Assisted Goal-Based Budget Planner
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool	tests/perf_load.py with HTTP requests, concurrent users and system resource sampling
Test Types	Functional testing and basic load testing
Main Areas	Planner endpoints, history access, validation, authentication and AI fallback behaviour
Test Mode	The included load evidence uses offline-fallback mode, so it measures application-side processing rather than live Gemini network latency
Evidence	6.Project Testing/evidence/

Step 2: Test Scenarios

S.No	Scenario	Virtual Users	Expected Result
1	Single-user baseline	1	Requests complete without errors
2	Normal classroom/demo load	10	Stable response times and no functional errors
3	Higher local load	25	Application remains responsive with a low error rate
4	Functional regression	1	Authentication, validation, planners, history and fallback tests pass

Step 3: Included Load Evidence

The repository contains a recorded 25-user run with the following values:

Metric	Recorded Value
Requests	2,724
Average response time	0.104 s
P95 response time	0.387 s
Maximum response time	2.055 s
Throughput	135 req/s
Error rate	0.00 %
Server RSS peak	91 MB

These figures describe the included **offline-fallback** run and should not be interpreted as live Gemini latency.

Step 4: Observations

- Application-side validation, calculation and SQLite operations are lightweight compared with an external AI request.

- The highest recorded latency occurs during bursts that include authentication work and other concurrent activity.
- Live Gemini latency depends on network conditions, API service response time and the selected model, so it must be measured separately before a production deployment.
- The next scaling steps are multiple workers, PostgreSQL, shared session storage and caching.

Step 5: Evidence

The raw files are stored in the repository under `6.Project Testing/evidence/`, including the load-test outputs and pytest result file. The functional test evidence should be rerun by the team after any final code changes.